HTTP & HTTPS

The method says what to do, the status says what happened

	GET	POST	PUT	PATCH
Safe	✓ yes	✗ no	✗ no	✗ no
Idempotent	✓ yes	✗ no	✓ yes	✗ no

Safe: only reads. **Idempotent**: repeating has the same effect. **PATCH**: not guaranteed.

1xx	2xx	3xx	4xx	5xx
info	**success**	**redirect**	**client error**	**server error**
100 101	200 201 204	301 302 304	401 403 404	500 502 503

1 Request

```
POST /items HTTP/1.1
Content-Type: application/json
{"name":"a"}
```

2 Response

```
HTTP/1.1 201 Created
Location: /items/7
{"id":7}
```

HTTPS = HTTP inside TLS — encryption · integrity · server identity

TRAP What happens here?

```
// POST /login answers:
HTTP/1.1 302 Found
Location: /home
```

`GET /home`

The body is dropped. **307** would repeat the POST.

KEY TAKEAWAY

- **Method** — The intent
- **Status** — The outcome
- **TLS** — The envelope

? Follow-up questions to expect

- **PUT or PATCH?** → PUT replaces the whole resource. PATCH sends only the changes.
- **401 or 403?** → 401: no valid credentials. 403: known user, not allowed.
- **Does `fetch` reject on 404?** → No. It resolves. Check response.ok.
- **What does HSTS do?** → Browser uses only HTTPS for that host.